



Ansible Automation Showcase

A practical reference for Ansible, AWX, Kubernetes, Argo CD, Terraform, Docker, and GitHub Actions.

Built from the current repository configuration, with setup instructions, architecture explanations, operating procedures, troubleshooting guidance, and learning exercises.

REFERENCE + LEARNING EDITION

Repository state documented August 2026



HOW TO USE THIS GUIDE

Contents

Section	Topic
1	System overview and current status
2	Repository structure
3	Ansible foundations and playbooks
4	AWX deployment model
5	Kubernetes workload
6	Argo CD GitOps configuration
7	GitHub Actions delivery pipeline
8	Terraform and Docker labs
9	End-to-end setup runbook
10	Operations, security, and troubleshooting
11	Learning path and command reference

Document scope

This guide documents what is present in the repository and clearly marks external prerequisites. It does not assume that AWX or Argo CD is already installed, reachable, or authenticated.

Current readiness

Area	Repository state	External requirement
Ansible	Playbooks, inventory, role, collection requirements, and root ansible.cfg exist.	Reachable managed hosts and valid credentials.
AWX	Operator kustomization and AWX custom resource exist.	Kubernetes cluster, operator installation, storage, ingress/TLS, credentials, and production hardening.
Argo CD	AppProject, Application, Kustomize files, and automated sync policy exist.	Argo CD installed and bootstrap manifests applied.
GitHub Actions	Validation and deployment workflow exists and runs on GitHub-hosted runners.	Production environment secrets and network reachability to Argo CD.
Terraform/Docker	Small learning examples exist.	Local tools and a suitable test environment.

Source basis

Generated from the repository files on branch main after commit 4dacadf. External systems such as Kubernetes, AWX, Argo CD, and GitHub secrets must be verified independently because their runtime state is not fully represented by source code.

ARCHITECTURE

1. System overview and current status

The repository combines configuration management and GitOps. Ansible manages operating systems and services. Argo CD continuously reconciles Kubernetes resources from Git. GitHub Actions validates proposed manifests and can request an exact-revision Argo CD sync.

End-to-end flow

Step	Component	Responsibility
1	Engineer	Changes Ansible, Argo CD, or Kubernetes code and opens a pull request.
2	GitHub Actions	Validates GitOps manifests with kubeconform on an Ubuntu runner.
3	Repository main	Stores the approved desired state and exact commit history.
4	Argo CD	Reads gitops/hello-world, calculates drift, applies changes, prunes removed objects, and self-heals drift.
5	Kubernetes	Runs two hello-kubernetes pods and exposes them through a NodePort Service.
6	Actions logs	Shows sync result, health, operation status, history, resources, and recent pod logs.

Two control planes

- Ansible and AWX are imperative orchestration tools: a job runs tasks against selected hosts.
- Argo CD is a declarative reconciler: it continuously compares Kubernetes with Git and corrects drift.
- GitHub Actions is the policy and evidence layer: it validates changes and records deployment logs.

Important current limitation

The live GitHub-hosted validation runner works. The deployment stage intentionally fails until the GitHub production environment contains `ARGOCD_SERVER` and `ARGOCD_AUTH_TOKEN` and the Argo CD endpoint is reachable. This is a safe failure, not a false success.

MAP

2. Repository structure

```

ansible-automation-showcase/
|-- .github/workflows/gitops-delivery.yml
|-- ansible.cfg
|-- ansible/
|   |-- inventory.ini
|   |-- requirements.yml
|   |-- playbooks/
|-- argocd/
|   |-- project.yaml
|   |-- hello-world-application.yaml
|   |-- kustomization.yaml
|-- gitops/hello-world/
|   |-- deployment.yaml
|   |-- service.yaml
|   |-- kustomization.yaml
|-- terraform/main.tf
-- devops-labs/
    |-- lab1-docker/
    |-- lab3-awx/

```

Path	Purpose	Change impact
ansible/	Host automation, service configuration, monitoring, users, patch scans, and NGINX examples.	Run manually or through AWX; not part of the current GitOps workflow trigger.
argocd/	One-time bootstrap and policy boundary for the Argo CD application.	Triggers validation and deployment workflow.
gitops/	Desired Kubernetes workload state reconciled by Argo CD.	Triggers validation and deployment workflow.
terraform/	Learning example that provisions a namespace, NGINX Deployment, and Service in Minikube.	Requires terraform init/plan/apply; independent of Argo CD.
devops-labs/	Container and AWX operator experiments.	Run independently in a suitable lab cluster.

Configuration discovery

The root-level `ansible.cfg` is discoverable when Ansible or AWX runs from the checked-out project root. Ansible checks `ANSIBLE_CONFIG` first, then `./ansible.cfg`, `~/ansible.cfg`, and `/etc/ansible/ansible.cfg`.

```

[defaults]
deprecation_warnings=False

```

For learning, consider enabling deprecation warnings after legacy settings are corrected. Suppressing all warnings can hide future migration work.

CONFIGURATION MANAGEMENT

3. Ansible foundations and playbooks

Inventory and dependencies

The inventory defines a webservers group with one RHEL host at 192.168.1.25. It uses SSH user mderangula and a local private-key path. The collection requirements request ansible.posix version 1.5.0 or newer.

```
ansible-galaxy collection install -r ansible/requirements.yml
ansible-inventory -i ansible/inventory.ini --graph
ansible all -i ansible/inventory.ini -m ansible.builtin.ping
```

Playbook catalog

Playbook	Learning objective	Operational note
create_users.yml	Loops, defaults, groups, and user creation.	Contains named users and fixed UIDs; review before production.
disk_monitoring.yml	Facts, calculations, conditional blocks, logging, and API alerting.	Monitoring URL is a placeholder and failures are intentionally ignored.
healthcheck.yml	Custom module use and conditional reporting.	Requires an appservers inventory group and local library/app_health.py.
patch_scan.yml	DNF update discovery and fact-based reporting.	Designed for RHEL-family systems.
setup-nginx.yml	Local Jinja2 rendering and output verification.	Writes under /tmp and does not deploy a service.
install-nginx-new.yml	Package, files, templates, handlers, service, and firewall management.	Review handler naming and RHEL firewall assumptions.
test-connection.yml	AWX project sync smoke test.	Runs locally and only emits a debug message.
ansible-nginx-test/site.yml	Reusable role structure with role variables.	Preferred learning path for reusable automation.

Safe execution pattern

```
# Syntax first
ansible-playbook -i ansible/inventory.ini PLAYBOOK --syntax-check

# Preview host selection and task changes
ansible-playbook -i ansible/inventory.ini PLAYBOOK --list-hosts
ansible-playbook -i ansible/inventory.ini PLAYBOOK --check --diff

# Run with explicit scope
ansible-playbook -i ansible/inventory.ini PLAYBOOK --limit rhel10
```

DEEP DIVE

3.1 Ansible design lessons

Idempotency

Most built-in Ansible modules describe desired state. Re-running package, user, file, template, service, and firewall tasks should not report changes when the target already matches. Commands are different: they need `changed_when`, `creates`, `removes`, or another guard to remain idempotent.

Variables and secrets

- Move environment-specific values such as hosts, ports, thresholds, and URLs into `group_vars` or `host_vars`.
- Use AWX Credentials or Ansible Vault for passwords, tokens, and private values.
- Do not commit private keys. The inventory path references a key that must exist securely on the execution host.
- Prefer dedicated automation identities and minimum sudo permissions.

Roles

The NGINX role separates defaults, variables, tasks, handlers, templates, metadata, and tests. This structure is reusable and easier to test than a single large playbook. A production role should also include supported-platform metadata, argument validation, Molecule scenarios, and CI linting.

Suggested Ansible quality gate

```
python -m pip install ansible ansible-lint
ansible-galaxy collection install -r ansible/requirements.yml
ansible-lint ansible/playbooks
ansible-playbook -i ansible/inventory.ini  ansible/playbooks/test-connection.yml --syntax-check
```

Production improvement backlog

Priority	Improvement	Reason
High	Re-enable deprecation warnings.	Prevents hidden breakage during Ansible upgrades.
High	Replace hardcoded inventory and users with environment data.	Reduces accidental changes and improves portability.
High	Add ansible-lint and syntax checks for ansible/** changes.	Stops invalid automation before merge.
Medium	Add Molecule tests for the NGINX role.	Verifies idempotency and behavior in disposable systems.
Medium	Replace simulated monitoring URL with a credentialed integration.	Turns the example into an operational alert path.

CENTRALIZED ANSIBLE

4. AWX deployment model

AWX provides project synchronization, inventories, credentials, job templates, schedules, RBAC, and job logs around Ansible. This repository includes a lab deployment using the AWX Operator version 2.19.1 and an AWX custom resource named `awx` in namespace `awx`.

Lab bootstrap

```
kubectl create namespace awx
kubectl apply -k devops-labs/lab3-awx
kubectl apply -f devops-labs/lab3-awx/awx-instance.yaml
kubectl -n awx get pods
kubectl -n awx get awx awx
```

Repository project setup

AWX object	Recommended value
Project	Git URL for ansible-automation-showcase; branch main; update on launch enabled.
Inventory	Use managed AWX inventory or inventory source; do not depend on a developer laptop key path.
Credential	Machine credential containing SSH key and optional privilege escalation settings.
Execution Environment	Pin an image containing compatible ansible-core and ansible.posix.
Job Template	Select a playbook, inventory, credential, execution environment, limits, and survey variables.

Where `ansible.cfg` belongs

Project-specific configuration belongs at the Git repository root, as it is here. Shared execution defaults can be baked into `/etc/ansible/ansible.cfg` in the Execution Environment. Secrets never belong in `ansible.cfg`.

Enterprise hardening

- Use a supported Kubernetes platform, persistent storage, backups, ingress, TLS, and an external identity provider.
- Separate development and production organizations, inventories, credentials, and execution environments.
- Use instance groups and capacity controls for job isolation and scaling.
- Forward AWX audit and job logs to centralized observability and retain them according to policy.
- Treat the NodePort service in this repository as a lab setting, not an internet-facing production design.

DESIRED STATE

5. Kubernetes workload

The hello-world workload contains a Deployment and Service composed with Kustomize. Argo CD deploys them into namespace hello-world.

Resource	Configuration	Effect
Deployment	hello-kubernetes; replicas: 2	Maintains two pods for basic availability.
Container	paulbouwer/hello-kubernetes:1.10	Runs the sample application on container port 8080.
Service	hello-kubernetes-svc; NodePort	Maps service port 80 to pod port 8080.
Kustomization	deployment.yaml + service.yaml	Creates a deterministic render target for Argo CD and local validation.

Render and inspect

```
kubectrl kustomize gitops/hello-world
kubectrl apply --dry-run=client -k gitops/hello-world
kubectrl -n hello-world get deploy,pods,svc
kubectrl -n hello-world rollout status deploy/hello-kubernetes
kubectrl -n hello-world logs deploy/hello-kubernetes --tail=100
```

Production gaps in the sample workload

Gap	Recommended addition
Floating operational behavior	Pin an image digest and use an internal trusted registry.
No health probes	Add startup, readiness, and liveness probes for port 8080.
No resource controls	Add CPU and memory requests and limits.
NodePort exposure	Use ClusterIP behind an ingress/gateway with TLS and policy controls.
No workload security context	Run as non-root, drop capabilities, use read-only root filesystem where possible.
No disruption policy	Add a PodDisruptionBudget and topology spread or anti-affinity.
No network policy	Restrict ingress and egress to required peers.

CONTINUOUS RECONCILIATION

6. Argo CD GitOps configuration

The AppProject named portfolio is the policy boundary. It permits only this repository, only the in-cluster Kubernetes API, and only namespace hello-world. It allows Namespace, Deployment, and Service resources and warns about orphaned resources.

Application behavior

Setting	Value	Meaning
Source	main / gitops/hello-world	Tracks the workload manifests on the main branch.
Destination	in-cluster / hello-world	Deploys to the Argo CD cluster in a dedicated namespace.
Automated prune	true	Deletes managed objects removed from Git.
Self-heal	true	Corrects manual drift from Git state.
Allow empty	false	Prevents an empty source from deleting everything.
Apply out of sync only	true	Avoids unnecessary writes to already-synced resources.
Prune last	true	Applies desired resources before pruning old ones.
Retry	5 attempts, exponential backoff	Handles temporary reconciliation failures.
History	10 revisions	Keeps a bounded rollback history.

One-time bootstrap

```
kubectl apply -f argocd/project.yaml
kubectl apply -f argocd/hello-world-application.yaml

argocd app get hello-world
argocd app sync hello-world
argocd app wait hello-world --sync --health --timeout 600
```

The project must exist before the application references it. A platform team normally owns this bootstrap layer; application teams own content under gitops/.

OBSERVE AND RECOVER

6.1 Argo CD operating model

Normal deployment

- A reviewed commit lands on main.
- Argo CD detects the new revision through polling or a repository webhook.
- Argo CD renders the Kustomize path, compares live state, applies changes, and waits for health.
- GitHub Actions explicitly requests the same commit SHA and records evidence in the run log.

Useful commands

```
argocd app list
argocd app get hello-world --show-operation
argocd app resources hello-world
argocd app history hello-world
argocd app diff hello-world
argocd app logs hello-world --since-seconds 600
argocd app rollback hello-world REVISION_ID
```

Drift and rollback

With self-heal enabled, manual kubectl edits are reverted. The preferred rollback is a Git revert because Git remains the audit record. Argo CD rollback is useful during an incident, but the repository must then be reconciled to avoid the next automatic sync reapplying the undesired revision.

Repository connectivity

The repository is public today, so Argo CD can read it without credentials. If it becomes private, configure a read-only deploy key or repository credential template. Rotate credentials and restrict them to the smallest repository scope.

Network topology

Argo CD API exposure	Runner choice	Guidance
Public with trusted TLS	GitHub-hosted Ubuntu	Restrict API access, use a least-privilege token, and monitor authentication.
Private network only	Ephemeral self-hosted runner	Place the runner in a controlled network with outbound GitHub access and private Argo CD reachability.
No CI-to-cluster access	GitHub-hosted validation only	Let Argo CD pull automatically; keep deployment evidence in Argo CD and observability.

VALIDATION AND EVIDENCE

7. GitHub Actions delivery pipeline

Triggers

- Pull requests targeting main run when argocd/**, gitops/**, or the workflow changes.
- Pushes to main run for the same paths and include deployment.
- workflow_dispatch allows an authorized manual run.

Validation job

The job runs on ubuntu-latest with a 10-minute timeout. It checks out the repository using an immutable Checkout v5 commit and runs kubeconform 0.8.0 using an image pinned by digest. Strict validation is enabled; unknown custom-resource schemas are ignored so Argo CD CRs do not fail offline validation.

Deployment job

The job runs only outside pull requests, waits for validation, uses the production environment, and has a 20-minute timeout. It verifies required secrets, downloads Argo CD CLI v3.4.2, validates the checksum, synchronizes GITHUB_SHA, prunes removed resources, and waits up to 10 minutes for sync and health.

Evidence step

The final step runs even after failure. It requests application state, operation details, deployment history, managed resources, and pod logs from the last 10 minutes. Diagnostic commands use '|| true' so one missing evidence source does not hide the original deployment result.

Required GitHub configuration

Name	Type	Purpose
production	Environment	Deployment boundary; add branch rules and required reviewers where available.
ARGOCD_SERVER	Environment secret	Reachable Argo CD API hostname.
ARGOCD_AUTH_TOKEN	Environment secret	Dedicated token limited to sync and read access for portfolio/hello-world.
ARGOCD_INSECURE	Environment variable	Lab-only TLS bypass. Leave false in production.

INTERPRETATION

7.1 Pipeline outcomes and logs

Outcome	Meaning	Next action
Validation fails	Malformed or schema-invalid GitOps manifest.	Read kubeconform output, fix locally, and update the pull request.
Missing ARGOCD_SERVER	Production environment or server secret is absent.	Create/configure the GitHub environment; do not weaken the check.
Authentication failure	Token is invalid, expired, or under/over scoped.	Issue a dedicated automation token and verify RBAC.
Connection timeout	Runner cannot reach the Argo CD API.	Fix DNS/TLS/firewall or use an online self-hosted runner.
Sync failed	Argo CD could not apply the desired state.	Inspect operation and resource messages in the evidence step.
Health timeout	Resources applied but did not become healthy.	Inspect pod events, probes, image pulls, scheduling, and logs.
Success	Exact commit is synced and healthy.	Review evidence, deployment history, and application monitoring.

Current observed behavior

The GitHub-hosted runner successfully completed manifest validation. Deployment then failed in the explicit configuration check because ARGOCD_SERVER was missing. This confirms runner and validator functionality while correctly preventing a pretend deployment.

Recommended branch and environment policy

- Require pull requests, successful validation, and review before merging to main.
- Prevent direct pushes and force pushes to main.
- Require production reviewers for high-risk environments.
- Keep workflow permissions at contents: read unless a specific additional permission is required.
- Use CODEOWNERS for .github/workflows/**, argocd/**, and production gitops paths.
- Enable dependency updates and secret scanning; review pinned action and image updates.

SUPPORTING EXAMPLES

8. Terraform and Docker labs

Terraform

terraform/main.tf uses the HashiCorp Kubernetes provider with Minikube context. It creates namespace portfolio-apps, a two-replica NGINX Deployment using nginx:latest, and a NodePort Service.

```
cd terraform
terraform init
terraform fmt -check
terraform validate
terraform plan
terraform apply
```

Learning note: the provider is constrained to ~> 2.0, but nginx:latest is not immutable. Production code should pin provider and image versions deliberately, use a remote state backend, protect state, and separate environments.

Docker lab

The Dockerfile builds a small Python 3.9-slim application on port 8080. Docker Compose builds the web service and starts Redis. The Compose file maps both application and Redis ports to the Mac.

```
cd devops-labs/lab1-docker
docker compose build
docker compose up -d
docker compose ps
docker compose logs -f web
docker compose down
```

Production improvements

- Use a supported Python version pinned by digest and run as a non-root user.
- Add a health check, dependency locking, vulnerability scanning, and a .dockerignore file.
- Do not expose Redis publicly; keep it on an internal network and configure authentication as required.
- Add resource limits, structured logs, and graceful shutdown behavior.

FROM CLONE TO DEPLOYMENT

9. End-to-end setup runbook

Step 1 - Prepare tools

- Install Git, Python, Ansible, kubectl, Kustomize support, Terraform, Docker, and the Argo CD CLI.
- Confirm access to the target Linux host and Kubernetes cluster.
- Use separate lab and production credentials.

Step 2 - Clone and validate

```
git clone https://github.com/mahder05/ansible-automation-showcase.git
cd ansible-automation-showcase
ansible-galaxy collection install -r ansible/requirements.yml
kubectl kustomize gitops/hello-world >/dev/null
kubectl kustomize argocd >/dev/null
terraform -chdir=terraform fmt -check
```

Step 3 - Install platform components

- Install Argo CD in namespace argocd using your platform's supported deployment method.
- Optionally install AWX through the lab operator manifests, then harden it before real use.
- Expose Argo CD through trusted TLS or provide a private runner with network access.

Step 4 - Bootstrap Argo CD

```
kubectl apply -f argocd/project.yaml
kubectl apply -f argocd/hello-world-application.yaml
argocd app get hello-world
```

Step 5 - Configure GitHub

- Create environment production.
- Restrict deployments to main and add required reviewers where appropriate.
- Add ARGOCD_SERVER and ARGOCD_AUTH_TOKEN as environment secrets.
- Set ARGOCD_INSECURE only in a disposable lab with untrusted TLS.

Step 6 - Prove the flow

- Open a pull request changing gitops/hello-world and verify validation logs.
- Merge after review and watch Sync and verify Argo CD.
- Confirm the exact commit, Synced status, Healthy status, resources, and logs.
- Verify Kubernetes resources in namespace hello-world.

RUN SAFELY

10. Operations, security, and troubleshooting

Routine checks

Frequency	Check
Per change	Review diff, validation result, Argo CD sync/health, evidence logs, and application smoke test.
Weekly	Review failed workflows, Argo CD drift, orphan warnings, pod restarts, and pending updates.
Monthly	Rotate or review automation tokens, patch runners/tools, test restore procedures, and review RBAC.
Quarterly	Exercise rollback, disaster recovery, credential revocation, and platform upgrade procedures.

Security principles

- Git is the desired-state audit trail; require review and signed or verified commits where policy requires.
- Use least-privilege Argo CD project roles and short-lived or regularly rotated automation tokens.
- Keep private keys and tokens in AWX/GitHub secret stores, never in the repository or logs.
- Pin workflow actions and container images; verify downloaded binaries with checksums or provenance.
- Separate build, validation, and production credentials. Pull-request jobs must not receive production secrets.
- Use trusted TLS. ARGOC_D_INSECURE is a lab escape hatch, not a production default.

Troubleshooting decision table

Symptom	Likely cause	Check
Workflow stays queued	No matching runner or concurrency lock.	Runner labels/status and active gitops-production run.
Kubeconform fails	Invalid YAML/schema or unsupported kind.	File path, API version, indentation, and schema output.
Argo CD app not found	Bootstrap not applied or wrong server/token.	kubectl -n argocd get applications and argocd context.
Permission denied	AppProject or token RBAC restriction.	Project source/destination/resource allowlists and token role.
OutOfSync persists	Apply error, ignored diff, webhook delay, or target revision issue.	argocd app get/diff and resource messages.
Degraded	Kubernetes workload unhealthy.	kubectl describe, events, probes, scheduling, image pulls, and logs.
Ansible unreachable	Network, SSH key, user, host, or Python issue.	ansible -m ping -vvvv and direct ssh test.

PRACTICE

11. Learning path and command reference

Suggested learning sequence

Stage	Exercise	Proof
1. Ansible basics	Run inventory graph, ping, syntax check, and check mode.	Explain inventory, modules, idempotency, facts, handlers, and privilege escalation.
2. Reusable roles	Run the NGINX role twice in a disposable RHEL host.	Second run reports no unexpected changes.
3. Kubernetes	Render and apply hello-world manually in a lab namespace.	Two Ready pods and a reachable Service.
4. Argo CD	Bootstrap the AppProject/Application and create deliberate drift.	Argo CD detects and self-heals drift.
5. CI/CD	Open a pull request with a valid and then invalid manifest.	Understand validation logs and merge protection.
6. Operations	Break an image tag, observe Degraded, then revert Git.	Use evidence, events, logs, and Git rollback.
7. Hardening	Add probes, resources, security context, digest pinning, and policies.	Pass validation and document the risk reduction.

Quick command reference

```
# Ansible
ansible-inventory -i ansible/inventory.ini --graph
ansible-playbook -i ansible/inventory.ini PLAYBOOK --syntax-check
ansible-playbook -i ansible/inventory.ini PLAYBOOK --check --diff

# Kubernetes and Kustomize
kubectl kustomize gitops/hello-world
kubectl -n hello-world get all
kubectl -n hello-world get events --sort-by=.lastTimestamp

# Argo CD
argocd app get hello-world --show-operation
argocd app diff hello-world
argocd app sync hello-world --prune
argocd app wait hello-world --sync --health --timeout 600
argocd app logs hello-world --since-seconds 600

# GitHub Actions
gh run list --workflow gitops-delivery.yml
gh run watch RUN_ID --exit-status
gh run view RUN_ID --log-failed
```

Final checklist

- Repository validation is green.
- Argo CD is bootstrapped and the application is Synced and Healthy.
- Production environment secrets and approvals are configured.
- Runner network path and TLS trust are verified.
- Workload security, probes, resources, and observability meet the target environment standard.
- Rollback and recovery procedures have been tested, not only documented.